

Star Dataset: Report

Machine Learning 6 Star Class Prediction & Analysis



Jaral Arroyo-Jefferson

Period 2

Applied Data Science

Table Of Contents

Research Question	2
Star Dataset (DF)	2
Dataset Cleaning	2
Machine Learning	3
References	12

Research Question

The purpose of making the dataset is to prove that the stars follow a certain graph in the celestial space

Star Dataset (DF)

- Absolute Temperature (in K)
- Relative Luminosity (L/Lo)
- Relative Radius (R/Ro)
- Absolute Magnitude (Mv)
- Star Color (White, Red, Blue, Yellow, yellow-orange, etc.)
- Spectral Class (O, B, A, F, G, K, M)
- Star Type (Red Dwarf, Brown Dwarf, White Dwarf, Main Sequence, SuperGiants, HyperGiants)

Dataset Cleaning

This dataset did not contain any missing values, however, not all the columns were in numerical values so I had to complete the encoding.

df.dtypes		df.isna().sum()	
Temperature (K)	int64	Temperature (K)	0
Luminosity(L/Lo)	float64	Luminosity(L/Lo)	0
Radius(R/Ro)	float64	Radius(R/Ro)	0
Absolute magnitude(Mv)	float64	Absolute magnitude(Mv)	0
Star type	int64	Star type	0
Star color	object	Star color	0
Spectral Class	object	Spectral Class	0
dtype: object		dtype: int64	

Encoding:

The columns I had to encode were Star Color and Spectral Class. I began by determining the unique values for each column and then mapping each non-numerical value to a number. Then I applied the map to the original column, defining it as a new encoded column. To simplify the colors I mapped them to only those found on roygbiv.

```

import numpy as np
def str_cleanup(Str):
    Str = Str.capitalize() # Capitalize the first letter
    Str = Str.strip()      # Remove leading and trailing whitespaces
    Str = Str.replace('-', ' ') # Replace hyphens with spaces
    return Str

# Assuming X_train is your DataFrame and 'Star color' is the column containing color labels
df['Star Color Map'] = np.vectorize(str_cleanup)(df['Star color'])

```

```
[ ] df['Star Color Map'].unique()
```

```

array(['Red', 'Blue white', 'White', 'Yellowish white',
      'Pale yellow orange', 'Blue', 'Whitish', 'Yellow white', 'Orange',
      'White yellow', 'Yellowish', 'Orange red'], dtype=object)

```

```

[ ] color_map = {
    'Red':0, 'Blue white':1, 'White':2, 'Yellowish white':3,
    'Pale yellow orange':4, 'Blue':5, 'Whitish':6, 'Yellow white':7, 'Orange':8,
    'White yellow':9, 'Yellowish':10, 'Orange red':11
}

```

```
[ ] df['Spectral Class'].unique()
```

```

array(['M', 'B', 'A', 'F', 'O', 'K', 'G'], dtype=object)

```

```
[ ] Spectral_Class_Map = { 'M': 0, 'B': 1, 'A':2, 'F':3, 'O':4, 'K':5, 'G':6}
```

```
[ ] df['Spectral Class Map'] = df['Spectral Class'].map(Spectral_Class_Map)
```

```

[ ] color_map_plot = { 'Red':'red', 'Blue white':'white', 'White':'white', 'Yellowish white':'white',
    'Pale yellow orange':'orange', 'Blue':'blue', 'Whitish':'white', 'Yellow white':'white', 'Orange':'orange',
    'White yellow':'yellow', 'Yellowish':'yellow', 'Orange red':'red' }

```

```
[ ] colors = df['Star color'].map(color_map_plot)
```

Machine Learning

Although this dataset already contained the Star type predictions I wanted to run my own model and compare my result to the listed value. For my machine learning program, I used the xgboost classification model. For my features, I used 'Temperature (K)', 'Luminosity(L/Lo)', 'Radius(R/Ro)', 'Absolute magnitude(Mv)', and 'Star Color Code'. After fitting the model I compared my model's predictions to the listed values of the dataset and had a 100% accuracy.

df.columns

```
Index(['Temperature (K)', 'Luminosity(L/Lo)', 'Radius(R/Ro)',  
      'Absolute magnitude(Mv)', 'Star type', 'Star color', 'Spectral Class',  
      'Star Color Map', 'Star Color Code', 'Spectral Class Map'],  
      dtype='object')
```

```
[20] import xgboost as xgb
```

```
[21] features = ['Temperature (K)', 'Luminosity(L/Lo)', 'Radius(R/Ro)', 'Absolute magnitude(Mv)',  
              'Star Color Code']
```

```
[22] X = df[features]  
     y = df['Star type']
```

```
# Initialize XGBoost model  
model = xgb.XGBClassifier()  
  
# Fit the model to the entire dataset  
model.fit(X, y)
```

```
XGBClassifier  
XGBClassifier(base_score=None, booster=None, callbacks=None,  
              colsample_bylevel=None, colsample_bynode=None,  
              colsample_bytree=None, device=None, early_stopping_rounds=None,  
              enable_categorical=False, eval_metric=None, feature_types=None,  
              gamma=None, grow_policy=None, importance_type=None,  
              interaction_constraints=None, learning_rate=None, max_bin=None,  
              max_cat_threshold=None, max_cat_to_onehot=None,  
              max_delta_step=None, max_depth=None, max_leaves=None,  
              min_child_weight=None, missing=nan, monotone_constraints=None,  
              multi_strategy=None, n_estimators=None, n_jobs=None,  
              num_parallel_tree=None, objective='multi:softprob', ...)
```

```
[24] df['My Prediction'] = model.predict(X)
```

```
from sklearn.metrics import accuracy_score  
  
# Calculate accuracy using scikit-learn  
accuracy = accuracy_score(df['Star type'], df['My Prediction'])  
print("Accuracy:", accuracy)
```

```
Accuracy: 1.0
```

Data Analysis & Visualization

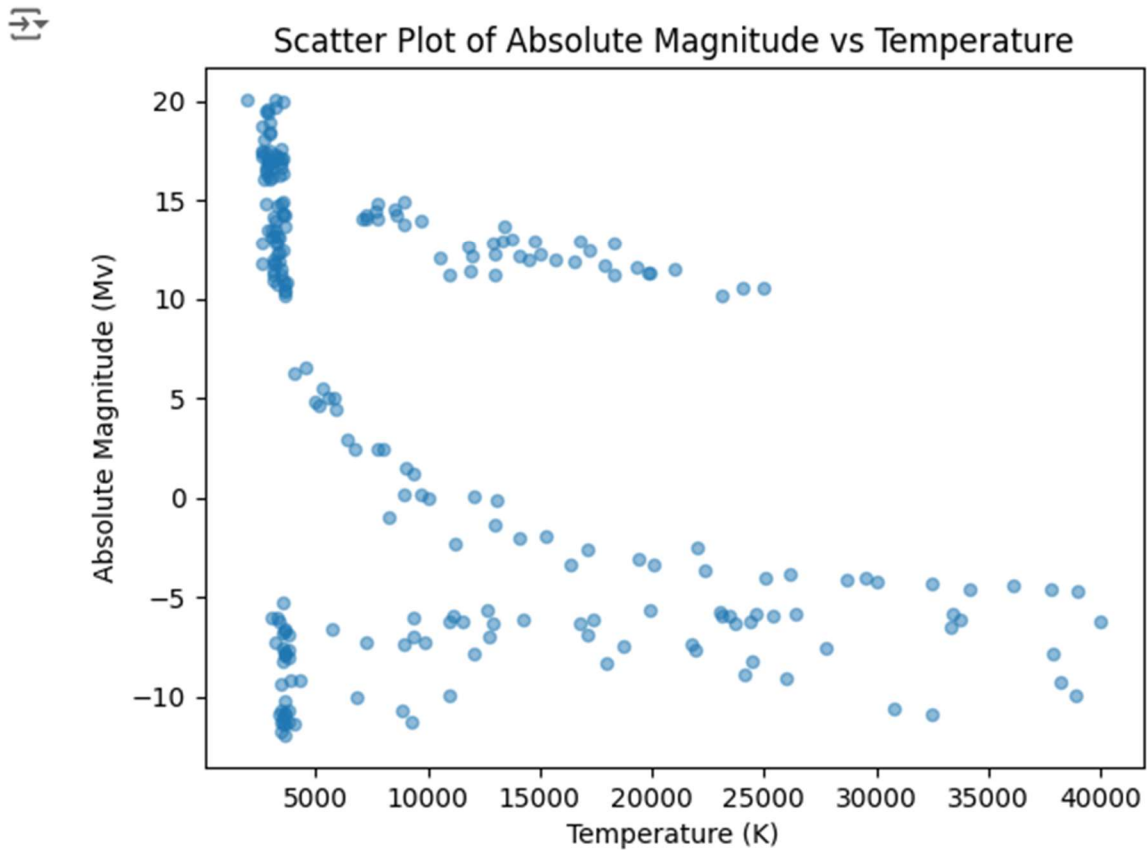
Google Colab:

The goal of my data analysis was to find the defining qualities of various star groups.

```
# Create a scatter plot
df.plot.scatter(x='Temperature (K)', y='Absolute magnitude(Mv)', alpha=0.5)

# Add labels and title
plt.title('Scatter Plot of Absolute Magnitude vs Temperature')
plt.xlabel('Temperature (K)')
plt.ylabel('Absolute Magnitude (Mv)')

# Show the plot
plt.show()
```




```

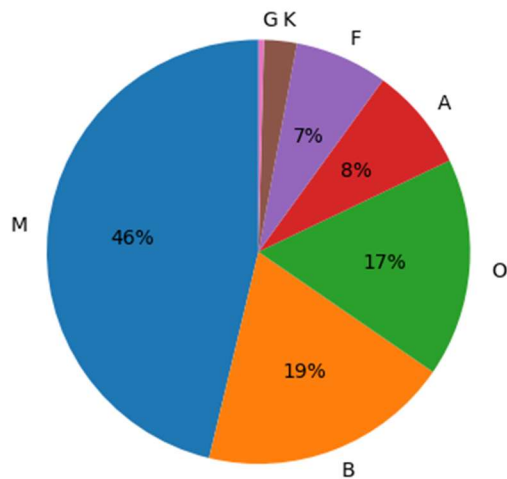
# Counting the frequency of each Spectral Class
class_distribution = df['Spectral Class'].value_counts()

# Plotting pie chart
class_distribution.plot(kind='pie', autopct=lambda p: '{:.0f}%'.format(p) if p > 5 else '', startangle=90)
plt.title('Distribution of Spectral Classes')
plt.ylabel('')
plt.show()

```



Distribution of Spectral Classes



```

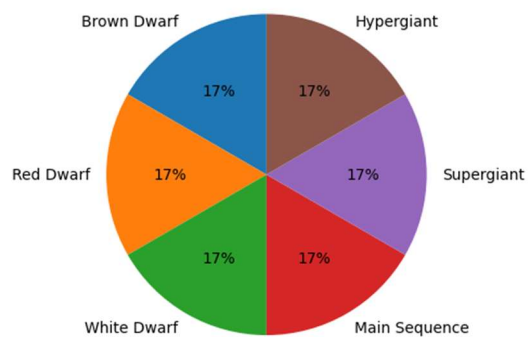
# Counting the frequency of each MP Star Type String
class_distribution = df['MP Star Type String'].value_counts()

# Plotting pie chart
class_distribution.plot(kind='pie', autopct=lambda p: '{:.0f}%'.format(p) if p > 5 else '', startangle=90)
plt.title('Distribution of MP Star Type Stringes')
plt.ylabel('')
plt.show()

```



Distribution of MP Star Type Stringes



ChatGPT:

ChatGPT is a great resource to be able to quickly develop simple visualisations for your data. Feeding it a prompt similar to the one below where you explain what your dataset is about, what type of graph you want, and your columns, ChatGPT is able to develop sophisticated graphs and charts. However there are some downsides. For example because I am not able to upload my entire Google Colab document, meaning ChatGPT does not know how the rest of my code looks, there may be issues when you run the generated code. Additionally, when you rely too heavily on ChatGPT for generating the code for the visualization you do not learn how the matplotlib library actually works and are simply a copy-and-paste bot.



You

can you develop some graphs using matplotlib python to analyse the columns of this star classification dataset with the following columns ['Temperature (K)', 'Luminosity(L/L_o)', 'Radius(R/R_o)', 'Absolute magnitude(M_v)', 'Star type', 'Star color', 'Spectral Class,']



ChatGPT

Sure, let's create some graphs to analyze the star classification dataset using matplotlib in Python. We can start with some basic visualizations like histograms and scatter plots to understand the distributions and relationships between the columns. Here's a Python script to get you started:

```
# Scatter plot: Absolute Magnitude vs Temperature for all stars
plt.figure(figsize=(8, 6))
plt.scatter(temperature, absolute_magnitude, alpha=0.5, color='blue')
plt.title('Absolute Magnitude vs Temperature for All Stars')
plt.xlabel('Temperature (K)')
plt.ylabel('Absolute Magnitude (Mv)')
plt.grid(True)
plt.show()
```

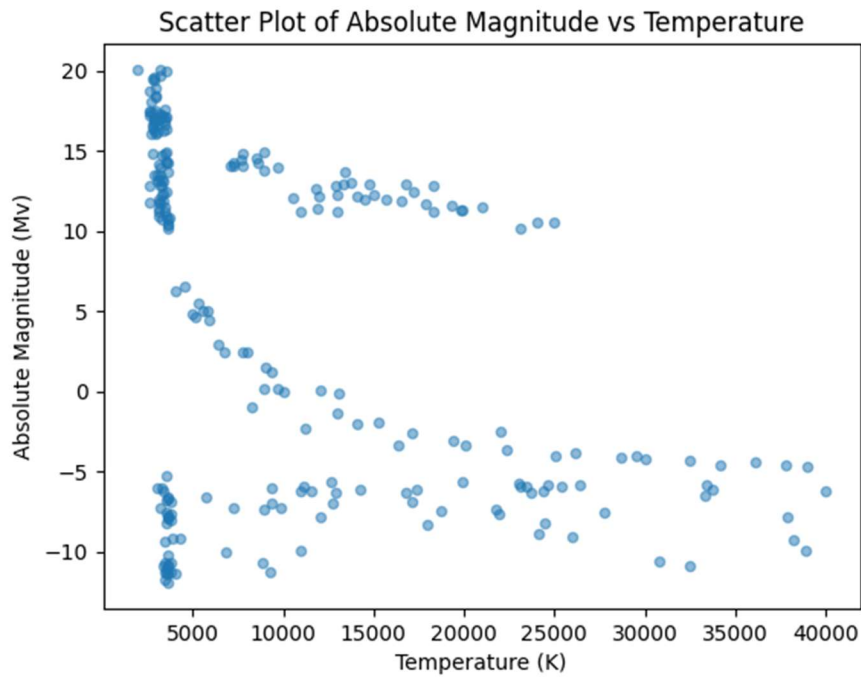
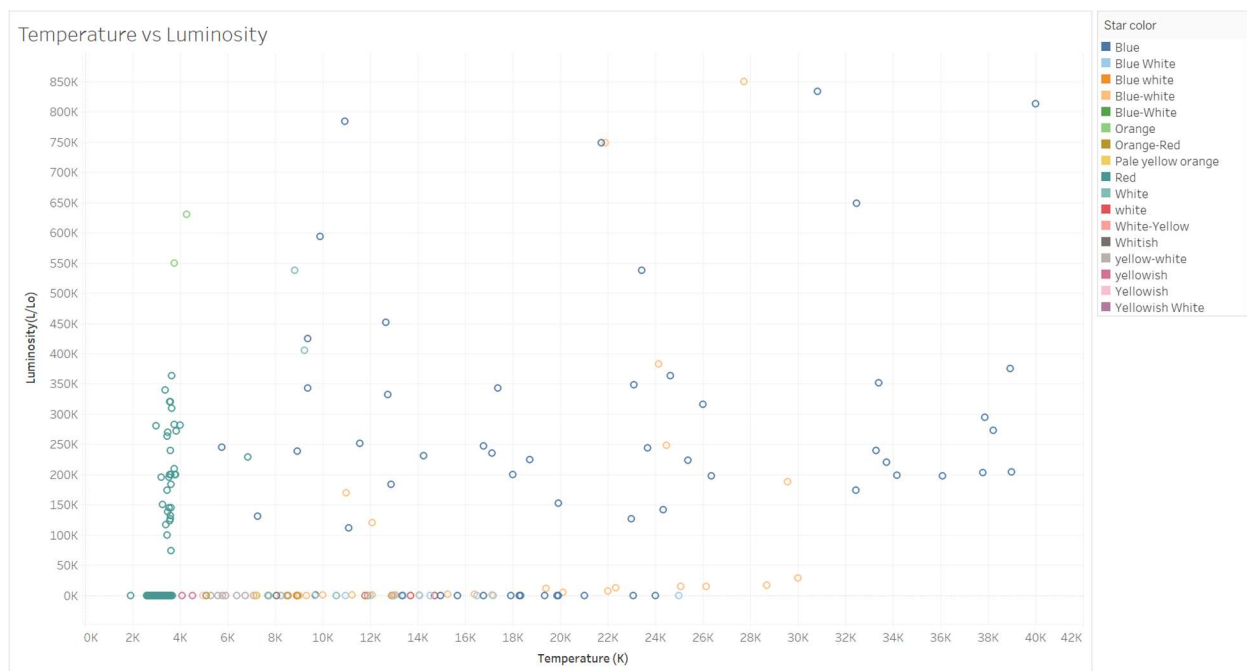
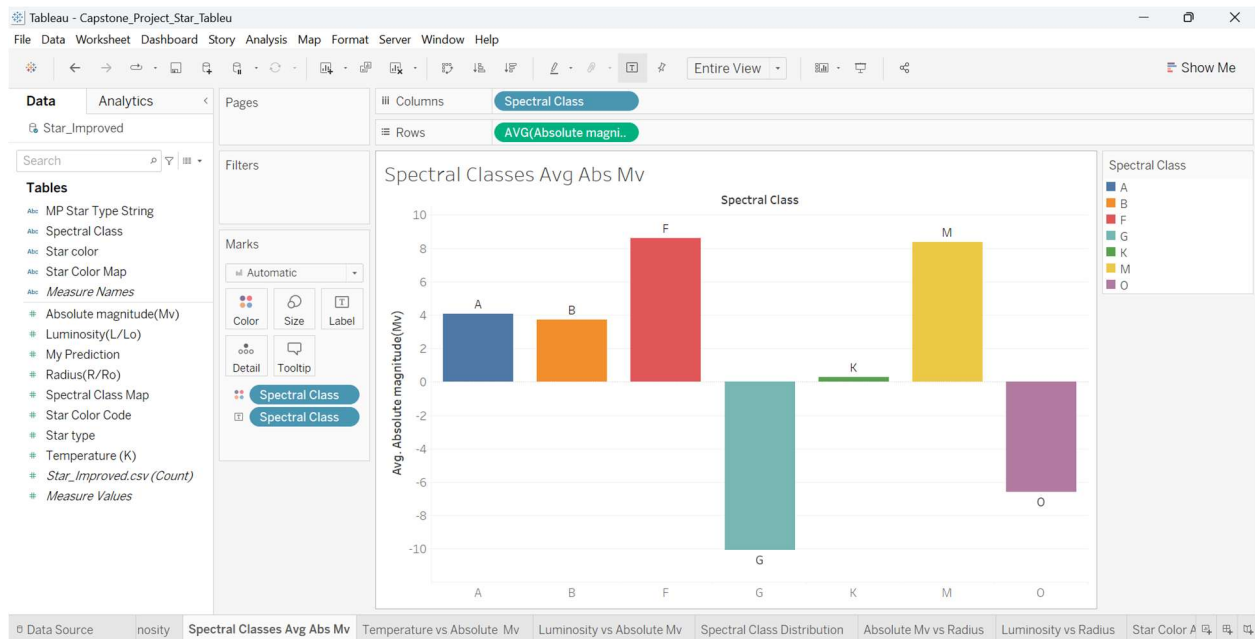
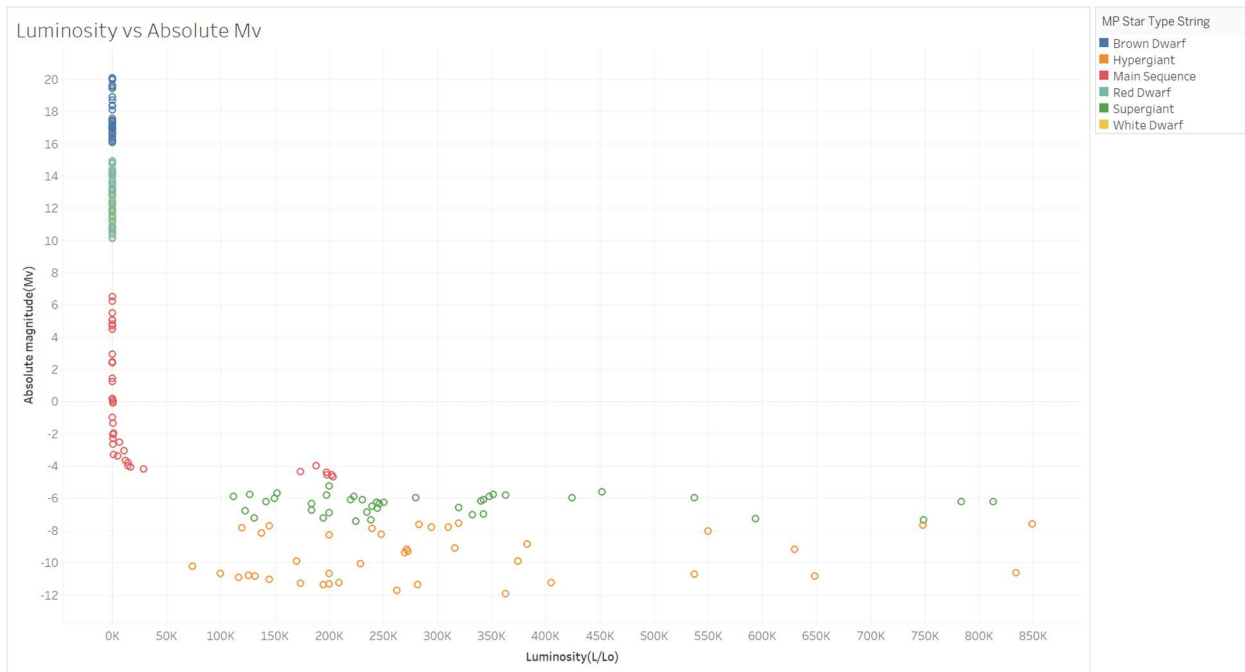



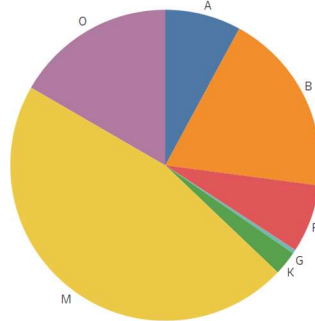
Tableau:

An even more efficient way to make quality data visualizations is using the Tableau software which allows for better customization. Here are some visualizations I made.





Spectral Class Distribution



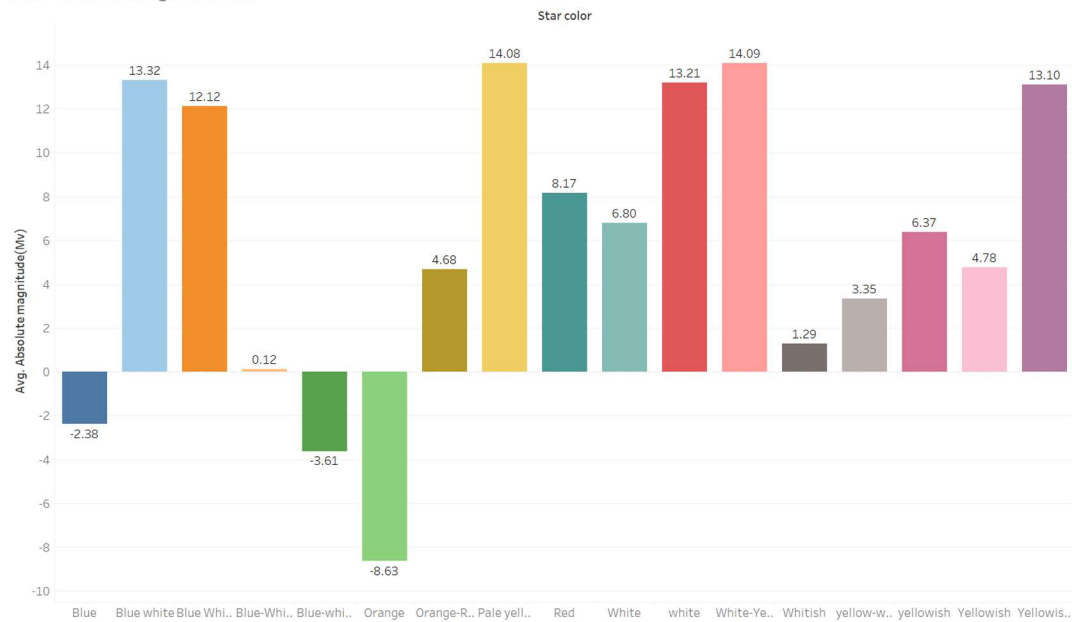
Spectral Class



CNT(Spectral Class)



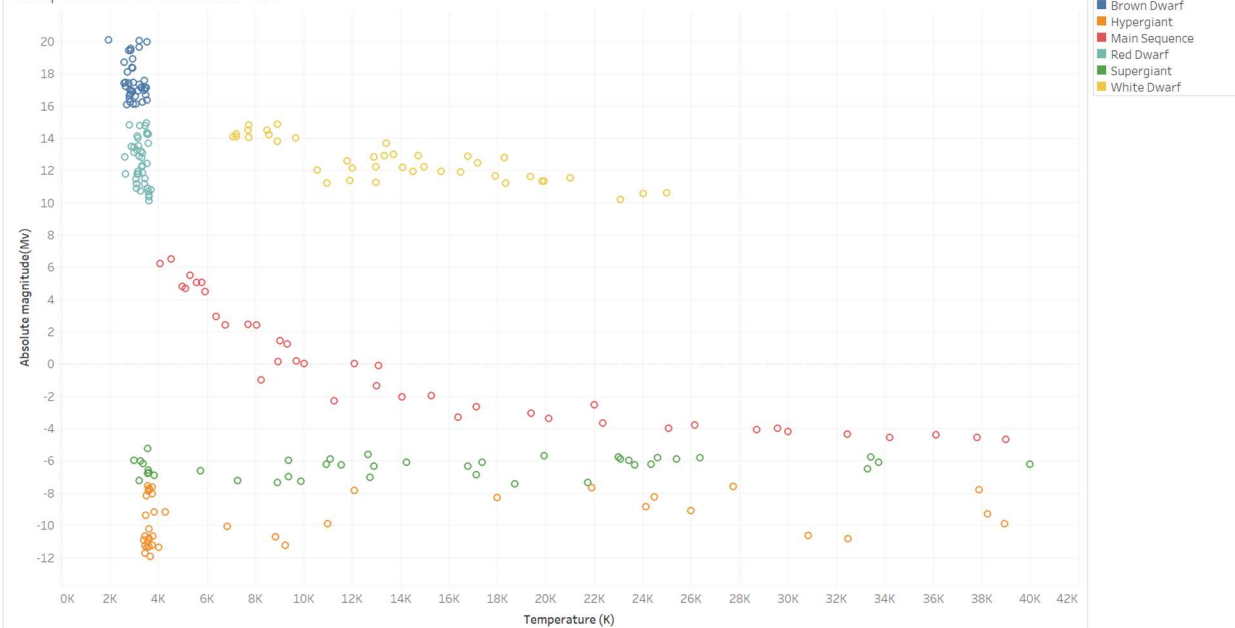
Star Color Average Abs. Mv



Star color



Temperature vs Absolute Mv



Concluding Thoughts

In this data science journey, I delved into a comprehensive stars dataset to uncover the scientific qualities underpinning star types. My objective was to decipher the intrinsic characteristics that determine the classification of stars, contributing to a deeper understanding of celestial dynamics.

Through analysis and visualization, I explored key attributes such as temperature, luminosity, radius, and absolute magnitude. Leveraging advanced machine learning models, I achieved 100% accuracy in predicting star types, highlighting the predictability and structure inherent in the dataset.

Visualization techniques revealed compelling relationships between luminosity and radius, as well as absolute magnitude and temperature. These insights shed light on the physical properties and evolutionary stages of stars, clarifying the scientific qualities that define their classifications.

Furthermore, the distribution of star types, colors, and spectral classes unveiled intriguing patterns indicative of underlying stellar phenomena. By unraveling these complexities, I have contributed to a deeper understanding of celestial dynamics and the scientific principles governing star classifications.

This study underscores the power of data science in unraveling the mysteries of the universe and advancing our scientific understanding of stellar phenomena.

References

1. <https://www.kaggle.com/datasets/deepu1109/star-dataset>